Chapter 4

Building Dynamic User Interfaces: DOM, Events, and AI Assistance in JavaScript

Table of Contents

Chapter 4 1

Building Dynamic User Interfaces: DOM, Events, and AI Assistance in JavaScript 1

4.1. Document Object Model (DOM) 2

4.1.1. More about the DOM 3

4.2. Working with the DOM tree 5

4.2.1. Accessing the elements 5

4.3. Working with DOM elements 10

4.3.1. How to get/update element content 11

4.3.2. Modifying text content 14

4.3.3. HTML DOM element styles 15

4.3.4. Working with classes 17

4.4. Parent/sibling/child 20

4.5. Creating Elements and Text Nodes 23

4.6. Removing Elements 26

4.7. Capturing Events with JavaScript 26

4.7.1. Ways to bind an event to an element 27

4.7.2. Event listeners 30

4.7.3. Mouse event 32

4.7.4. Keyboard events 36

4.7.5. DOM objects 36

4.7.6. Window event 44

4.7.7. Form events 54

4.1. Document Object Model (DOM)

The **Document Object Model** (*DOM*) is the data representation of the objects that comprise the structure and content of a document on the web. This guide will introduce the DOM, look at how the DOM represents an HTML document in memory and how to use APIs[1] to create web content and applications.

What is the DOM?

The Document Object Model (DOM) is a programming interface for web documents. It represents the page so that programs can change the document structure, style, and content. The DOM represents the document as nodes and objects; that way, programming languages can interact with the page.

A web page is a document that can be either displayed in the browser window or as the HTML source. In both cases, it is the same document but the Document Object Model (DOM) representation allows it to be manipulated. As an object-oriented representation of the web page, it can be modified with a scripting language such as JavaScript. (Introduction to DOM, 2021)

Objects Properties

The browser is an object, and the document it displays is an object too. The browser itself has a long list of objects including: the browser window, the document inside the window, the navigation buttons, the location or URL, and more. These objects are modelled by what's known as the Browser Object Model, or BOM for short.

Because all these things are just objects, we can interact with them using JavaScript the same way we interact with any other object. If you want to know the width of the current viewport, you can simply ask for the window object, and its inner width property: window.innerWidth. If you want to open a new tab you use the window open method. Window is the top-level object in the BOM, and it has a ton of properties and methods you can use to interact with the browser itself, and what it displays.

Document is one of the properties in the window object, which contains the current HTML document. Document has its own object model known as DOM. If you have worked with HTML and CSS before, you are already intimately familiar with the DOM,

[1] An **application programming interface** (**API**) is a connection between computers or between computer programs. It is a type of software interface, offering a service to other pieces of software.[1]

even if you did not know it existed. The Document Object Model is the model of the document that forms the current webpage.

In HTML, every piece of content is wrapped in a beginning and end tag creating an HTML element. Each of these elements is a DOM node, and the browser handles each of them the same way it would handle an object. That's why when you target something with a CSS rule, say all anchor tags within the document, that rule is implemented to each of them individually. When you write a CSS rule targeting a, you're saying: "find me every node, and apply the following style property settings to it". When a document is loaded in the browser, it is loaded into the document object in the Browser Object Model[2], BOM, and a Document Object Model is created for just this document instance. The browser now creates a node tree, modelling the relationships between the different nodes. In a standard HTML document you'll have an HTML object containing two nodes: head, and body. Head holds all the invisible objects like title, link, meta, script, etc., while body holds all the visible nodes in the viewport. JavaScript sees any webpage as an object tree.

4.1.1. More about the DOM

The Document Object Model (DOM) specifies how browsers should create a model of an HTML page and how JavaScript can access and update the contents of a web page while it is in the browser window.

The DOM is neither part of HTML, nor part of JavaScript; it is a separate set of rules. It is implemented by all major browser makers, and covers two primary areas:

- Making a model of the HTML page
    - When the browser loads a web page, it creates a model of the page in the memory.
    - The DOM specifies they way in which the browser should structure this model using a **DOM tree.**
    - The DOM is called an object model because the model (the DOM tree) is made of objects.

[2] In JavaScript, the **BOM (Browser Object Model)** allows JavaScript to interact with the web browser itself, outside of the webpage content. The BOM provides objects and methods to control browser features such as: Browser windows and tabs, The browser URL, Navigation history , Screen information, Alerts and dialogs, Timers. The main object in the BOM is the window object.

    - Each object represents a different part of the page loaded in the browser window.
- Accessing and changing the HTML page
    - The DOM also defines methods and properties to access and update each object in this model, which in turn updates what the user sees in the browser.
    - You will hear people call the DOM an Application Programming Interface (API). User interfaces let humans interact with programs; APIs let programs (and scripts) talk to each other. The DOM states what your script can ask the browser about the current page, and how to tell the browser to update what is being shown to the user.

Types of DOM tree nodes

The model DOM tree consists of four main types of nodes:

- The **document node** presents the entire HTML page.
- The **element nodes** are HTML elements in the HTML page.
- The **attribute nodes** carry attributes that are associated to the element. Attributes nodes are not children of the element that carries them; they are part of that element. Once you access the element, there are specific JS methods and properties to read or change that element's attributes.
- The **text nodes** are the text in an element. Once you have accessed an element node, you can then reach the text within that element. This is stored in its own text node. Text nodes cannot have children. If an element contains text and another child element, the child element is not a child of the text node but rather a child of the containing element.

Let us have a look of the following HTML page in a DOM tree structure:

HTML Page

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <section class="container">
```

```
        <h1 class="title">Welcome to JavaScript programming</h1>
        <h2>Learning objectives:</h2>
        <ul>
            <li>Translate design direction into a responsive
             implementation using different cloud client technology.
            </li>
            <li>Build and test code using text editor and translate
            design instruction into code.
            </li>
        </ul>
    </section>
 </body>
</html>
```

DOM Tree Structure

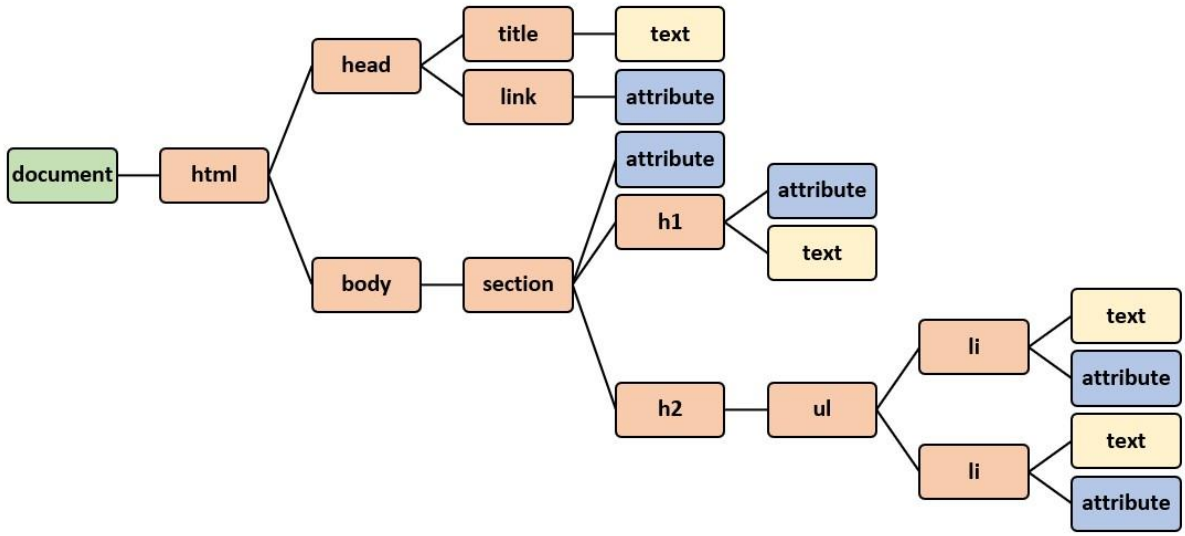


Diagram 1 – DOM structure of a basic webpage

4.2. Working with the DOM tree

Accessing and updating the DOM tree involves two steps:

1. Accessing the elements
2. Working with DOM elements

4.2.1. Accessing the elements

DOM queries may return one element, or they may return a **NodeList**, which is a collection of nodes.

- *getElementById* or *querySelector* method access to one individual element (or a fragment of the page that is stored within that one element).
- *getElementByClassName, getElementsByTagName, or querySelectorAll* method select a group of elements. These group of methods return more than one node, it will always return a **NodeList**, which is a **collection** of nodes (even if it only finds one matching element). You then need to select the element you want from the list using an index number (which means the numbering starts at 0 like the items in an array).

Traditional methods getElementById, getElementByClassName, getElementsByTagName methods work fine, but they're often too specific and a bit clunky to work with. Especially if you're looking for a node inside a node inside a node. More recently, two new catchall methods have come along to solve pretty much all our targeting needs: querySelector, which returns the first instance that matches the specified selectors, and querySelectorAll which returns a node list of all elements that match the specified selectors. These selectors are one or more comma separated CSS selectors, and they can target elements within the document the same way you would target them using a style rule. That makes Query Selector and its sibling both easy to work with, and incredibly powerful.

There are still cases where you might want to use one of the traditional methods. Specifically if you're working with forms, or if you want your code to be unambiguous. But for most uses, **querySelector**, and **querySelectorAll** is the way to go.

Document Methods

JavaScript uses dots(.) to separate objects from their properties or methods. Let us see some of the JS methods and dot uses.

`getElementById()`

The document method, **getElementById(),** will go into the document and look for a specific element.

```
<p id="one" >Here is some text</p>
<script>
   document.getElementById('one').style.color ='red';
</script>
```

getElementsByTabName()

The **getElementsByTagName()** method returns a collection of all elements in the document with the specified tag name, that I can use the elements for something else later.

```
<p>Here is some text 1</p>
<p>Here is some text 2</p>
<p>Here is some text 3</p>
<script>
      let myText = document.getElementsByTagName('p');
      console.log(myText);
</script>
```

To access the elements in a collection, we can use a for loop.

```
<p>First Name </p>
<p>Last Name </p>
<p>Full Name </p>

<script type="text/javascript">
let parag = document.getElementsByTagName('p');
for (let i = 0; i<parag.length; i++){
      parag[i].style.color='green';
      alert(`showing paragraph ${i+1}`);
  }
</script>
```

getElementsByClassName()

The **getElementsByClassName()** method returns a collection of all elements in the document with the specified class name

```
<p class="name">First Name </p>
<p class="name">Last Name </p>
<p>Full Name </p>

<script type="text/javascript">
   let n = document.getElementsByClassName('name');
   for (let i = 0; i < n.length; i++) {
      n[i].style.color = 'blue';
   }
</script>
```

You can also get one element:

```
<p class="name">First Name </p>
<p>Last Name </p>
<p>Full Name </p>

<script type="text/javascript">
   let n = document.getElementsByClassName('name');
   n[0].style.color = 'blue';
</script>
```

querySelector()

The **querySelector()** method returns the first element that matches a specified *CSS selector(s)* in the document. It can selects by using any method, by class name, by id name, or by element name. querySelector() only return the first matching of the selection.

```
<div id="special">
   <p >First Name</p>
   <p>Last Name</p>
   <p class="n1">Full Name</p>
</div>

<script type="text/javascript">
   let myText = document.querySelector('#special .n1');
   myText.style.color = 'red';
   let firstP = document.querySelector('p');
   firstP.style.backgroundColor = 'orange';
 </script>
```

querySelectorAll();

The **querySelectorAll()** method returns all elements in the document that matches a specified CSS selector(s), as a static NodeList object.

The NodeList object represents a collection of nodes. The nodes can be accessed by index numbers. The index starts at 0.

```
<div id="special">
    <p class="n1">First Name</p>
    <p>Last Name</p>
    <p>Full Name</p>
</div>
```

```
<script type="text/javascript">
    let myText = document.querySelectorAll('.name');
    for(var i=0; i<myText.length; i++){
      myText[i].style.fontWeight = "bold";
    }
</script>
```

Example 1) use querySelector and querySelectorAll to select different elements in a DOM

HTML code

```
<h1 id="title">Accessing elements</h1>
<p class="description">
      Traditional methods to access elements:
</p>
<ul class="traditionalMethods">
     <li class="methods">getElementById</li>
     <li class="methods">getElementsByClassName</li>
     <li class="methods">getElementsByTagName</li>
</ul>
<p class="description"> Most recent methods to access elements: </p>
<ul class="recentMethods">
     <li class="methods">querySelector</li>
     <li class="methods">querySelectorAll</li>
</ul>
```

JS code

```
// select element with class name "description"
let desc = document.querySelector(".description");
console.log(desc);
// select element with id name "title"
let titleNode = document.querySelector("#title")
// select element with tag "p"
 let par = document.querySelectorAll("p")
 console.log(par)
 // select all elements with class name "methods"
 let methods = document.querySelectorAll(".methods")
 console.log(methods)
 console.log("Loop through each element in the nodeList")
 let lengthNode = methods.length
 for(let i = 0; i<lengthNode;i++){
      console.log(methods[i],"\t",methods[i].textContent) }
```

4.3. Working with DOM elements

There are different ways to access/update element content once the element is selected. Your choice of techniques depends upon what the element contains.

Before we move to learn the different techniques to access/update elements, we are going to learn how to apply a click event to an element. We are learning a click event first because we want to see the changes or updates to the element when it's clicked.

Everything that happens in the browser is registered as an event. Open the browser, that's an event. Enter a URL in the address bar and hit enter, that's an event. Click a link, that's an event. Move your mouse, that's also an event. Whenever you interact with the browser in any way, you fire an event and the browser reacts to that event with preconfigured responses. Using JavaScript, we can write our own scripts to respond to these events as well.

The formula for building JavaScript functionality around events is called `addEventListener().` The `addEventListener()` method attaches an event handler to a document. and it typically takes this form:

- Add a click event to the document:

The JavaScript to add click event to an element will be as:

```
document.addEventListener("click", myFunction);
function myFunction() {
  console.log("click event")
}
```

- Simpler syntax:

```
document.addEventListener("click", function(){
  console.log("click event")
});
```

Now that we know how to add an event to an element, let's see how we can work with HTML elements.

4.3.1. How to get/update element content

To work with the content of an elements we can:

- Navigate to the text nodes and save the node in a variable.
- Work with the containing elements. This allows you to access its text nodes and child elements. It works better when an element has text nodes and child elements that are siblings.

Text nodes

Once you have navigated from an element to its text node, there is one property that you will commonly find yourself using is **nodeValue**, which allows you to access text from a node.

HTML code

```
<li id="one"><em>emphasis</em> tag</li>
```

JS code

```
let li1 =
document.querySelector("#one").firstChild.nextSibling.nodeValue
console.log(li1)
```

Containing element

When you are working with an element node (rather than its text node), that element can contain markup. You must choose whether you want to retrieve (get) or update(set) the markup as well as the text. Those properties are *innerHTML, textContent,* and *innerText.* When you use these properties to update the content of an element, the new content will overwrite the entire contents of the element (both text and markup)

Class names

The **className** property sets or returns an element's class attribute.

Example 2) Create a mini app that changes the shape of a element whenever a user selects a shape using buttons.

HTML code

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
     initial-scale=1.0">
    <title>JavaScript </title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1 class="title">JavaScript by ________ </h1>
    <section class="shapeContainer">
        <div class="shapeDiv "><p>Change shape</p></div>
        <button class="btnsquare">Square</button>
        <button class="btnrectangle">Rectangle</button>
        <button class="btncircle">Circle</button>
    </section>


    <script src="script.js"></script>
</body>
</html>
```

CSS code

```
*{box-sizing: border-box;}
.title{text-align: center; color: magenta}
/*------ SHAPES ------- */
.shapeContainer{
    width: 500px;
    margin: 20px auto;
    background-color: azure;
    padding: 20px;
    text-align: center;
}
.shapeDiv{
    width: 200px;
    color: purple !important;
}
.square{
    width: 200px;
    background-color: blueviolet;
}
.rectangle{
```

```
    width: 400px;
    background-color:olive;
}
.circle{
    width: 200px;
    border-radius: 50%;
    background-color: crimson;
}
.square , .rectangle , .circle, .shapeDiv{
    height: 200px;
    font-size: 1.5em;
    color: white;
    text-align: center;
    padding-top: 10%;
    margin: 20px auto;
}
```

JS code

```
/* SHAPE */
// collect the elements
let shape = document.querySelector(".shapeDiv")
let btnsquare = document.querySelector(".btnsquare")
let btnrectangle = document.querySelector(".btnrectangle")
let btncircle = document.querySelector(".btncircle")
let displayshape = document.querySelector(".shapeDiv p")

// add a click event to each button
btnsquare.addEventListener("click", function(){
    shape.className= "square"
    displayshape.textContent = "square".toUpperCase()
})

btnrectangle.addEventListener("click", function(){
    shape.className = "rectangle"
    displayshape.textContent = "rectangle".toUpperCase()
})

btncircle.addEventListener("click", function(){
    shape.className = "circle"
    displayshape.textContent = "circle".toUpperCase()
})
```

4.3.2. Modifying text content

We can modify the text content of an element by using the properties **.innerText**, **textContent** , and **innerHTML**.

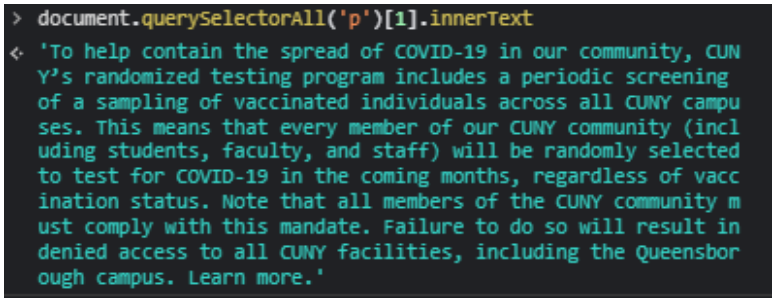
- **textContents** is all text contained by an element and all its children that are for formatting purposes only.
- **innerText** returns all text contained by an element and all its child elements.
- **innerHTML** returns all text, including html tags, that are contained by an element.

.innerText

Consider a paragraph in a document, as shown below:

```
> document.querySelectorAll('p')[1]
<- ▼<p>
     "To help contain the spread of COVID-19 in our community,
     "
     <a href="https://www.cuny.edu/coronavirus/covid-19-testin
     g-program-facultyandstaff-faq/#random-2" rel="noopener"
     target="_blank">CUNY's randomized testing program</a>
     " includes a periodic screening of a sampling of
     vaccinated individuals across all CUNY campuses. This
     means that every member of our CUNY community (including
     students, faculty, and staff) will be randomly selected
     to test for COVID-19 in the coming months, regardless of
     vaccination status. Note that all members of the CUNY
     community must comply with this mandate. Failure to do so
     will result in denied access to all CUNY facilities,
     including the Queensborough campus. "
     <a href="coronavirus/back-to-queensborough.html#testing">
     Learn more</a>
     "."
   </p>
```

If we select the innerText of the paragraph, it will display only the text content and exclude the hyperlink contained within the paragraph, as shown below:



innerHTML

Using the innerHTML property can be very powerful. The innerHTML property sets or returns the HTML content (inner HTML) of an element. It can change the complete content of an element.

Example 3) Change the text content of the second paragraph in the following HTML code by replacing it with new italicized text.

```
<div id="special">
    <p class="n1">Welcome to JS DOM</p>
    <p>Document Object Model</p>
    <p>JavaScript</p>
</div>
<script type="text/javascript">
    let myDiv = document.querySelectorAll('p')[1];
      myDiv.innerHTML='New Paragraph using innerHTML <em><b> Hey
there</b></em>';  </script>
```

4.3.3. HTML DOM element styles

The **style** property returns the values of an element's style attribute. The style attribute is written in camel-case form, which is a writing phrase without space or punctuation instead with capitalized words. For example, attribute **font-color** in camel-case is **fontColor**.

The **style** property returns a **CSSStyleDeclaration** object.

The **CSSStyleDeclaration** object contains all inline styles properties for the element. It does not contain any style properties set in the <head> section or in any external style sheets.

Example 4) add different styles to the DOM elements in Example 1

```
let pContent = document.querySelector(".content")

let b1 = document.querySelector(".button1")

let b2 = document.querySelector(".button2")
let b3 = document.querySelector(".button3")
b1.addEventListener('click', function(){
     pContent.textContent = "Number 1"
     pContent.style.backgroundColor = "lightblue"
     pContent.style.padding = "1em 2em"
})
b2.addEventListener('click', function(){
     pContent.innerText = "Number 2"
     this.style.color = "magenta"
     this.style.fontSize ="2em"  ///
})
b3.addEventListener('click', function(){
     pContent.innerHTML = "<h1>Number 3</h1>"
})
```

Example 5) Apply different colors from the colors array to each **** element so that, when combined, they display the word “RAINBOW” in multiple colors.

```
<h1>
      <span>R</span>
       <span>A</span>
       <span>I</span>
       <span>N</span>
       <span>B</span>
       <span>O</span>
       <span>W</span>
</h1>

const colors = ['red', 'orange', 'yellow', 'green', 'blue', 'indigo',
'violet']; //PLEASE DON'T CHANGE THIS LINE!
const letters = document.querySelectorAll('span');
let counter = 0
for (let eachLetter of letters) {
     eachLetter.style.color = colors[counter]
     counter++  }
```

4.3.4. Working with classes

className property

The className property sets or returns the class name of an element (the value of an element's class attribute). For this, the class name should have been set in <style> or CSS file

Example 6) add a class **.colorblue** to the first paragraph in the following HTML code:

```
<!DOCTYPE html>
<html>
<head>
<style>
  .colorblue{color:blue;}
</style>
</head>
<body>
  <div class="special">
    <p>Here is some text</p>
    <p>Here is some more text</p>
  </div>

  <script type="text/javascript">
    let firstPara = document.querySelector('p');
    firstPara.className="colorblue";
  </style>
```

setAttribute()

The setAttribute(attribute's name, attribute's value) method adds the specified attribute to an element and gives it the specified value. If the specified attribute already exists, only the value is set/changed.

Example 7) select the input checkbox and set it as checked

```
<body>
<form>
  <label><input type="checkbox">YES</label>
</form>

<script type="text/javascript">
  let myCheckbox = document.querySelector('input');
  myCheckbox.setAttribute('checked', 'checked');
</script>
```

Example 8) Manipulating Attributes: using the DOM elements attributes, select the image and change its alt text to "image of egg"

```
<img src="https://devsprouthosting.com/images/egg.jpg" width="200px">
<script type="text/javascript">
   let image = document.querySelector('img');
   image.setAttribute('alt', 'chicken')
</script>
```

Adding classes

Adding a class using **setAttribute** will override the current class.

Example 9) add another class name to the <h2> element of the following HTML code:

```
<!DOCTYPE html>
<head>
    <title>Forest</title>
      <style media="screen">
    .subtitle{ text-align: center; color: green; }
    .colorSubtitle{color: red; text-align: right; font-family:algerian}
    </style>
</head>
<body>
  <!-- example adding classes -->
  <h2 class=" colorSubtitle ">Example of classes</h2>
  <script type="text/javascript">
    // adding classes
    document.querySelector('h2').setAttribute('class','subtitle')
  </script>
</body>
</html>
```

Instead of overwriting the current class, we can use the **classList.add()** to add a class to an element:

```
document.querySelector('h2').classList.add('subtitle')
```

If we check on the console, we can see that <h2> now has two classes:

```
<h2 class="colorSubtitle subtitle">Example of classes</h2>
```

Removing class name from an element

We can also remove a class by using the .**classList.remove().** For example, if we want to remove class colorSubtitle from the <h2> element.

```
document.querySelector('h2').classList.remove('colorSubtitle')
```

We can use `.classList.toggle('className')` to add or remove a class

Example 10) use JS to invert the class in each element

```
<style>
  li { background-color: #B10DC9; }
  .highlight {  background-color: #7FDBFF;}
</style>
<ul>
      <li>Hello</li>
      <li class="highlight">Hello</li>
      <li>Hello</li>
      <li>Hello</li>
      <li class="highlight">Hello</li>
      <li>Hello</li>
  </ul>
```

The unordered list should appear as shown below:

- Hello
- Hello
- Hello
- Hello
- Hello
- Hello

By toggling the class name of each list item using the highlight class:

```
<script>
  const listItem = document.querySelectorAll('li');
      for(let eachLi of listItem){
      eachLi.classList.toggle('highlight')
    }
</script>
```

The class will toggle, meaning the list item will change from the highlight class to having no class name, and vice versa.



4.4. Parent/sibling/child

Parent elements can traverse upwards of an element. We can find the parent element by using the property **.parentElement**. Every element can only have one direct parent but can have multiple children.

Example 11) find the relationship of each element on the following HTML code

```
<!DOCTYPE html>

<head>
    <title>DOM example</title>
 </head>

<body>
<p>Fruit Shopping List</p>
  <ul>
      <li>Apple</li>
      <li class="highlight">Orange</li>
      <li>Pearl</li>
      <li>Grapes</li>
      <li class="highlight">Strawberries</li>
      <li>Watermelon</li>
  </ul>
```

For the HTML code above, we can identify the parent element of the list items using the parentElement property.

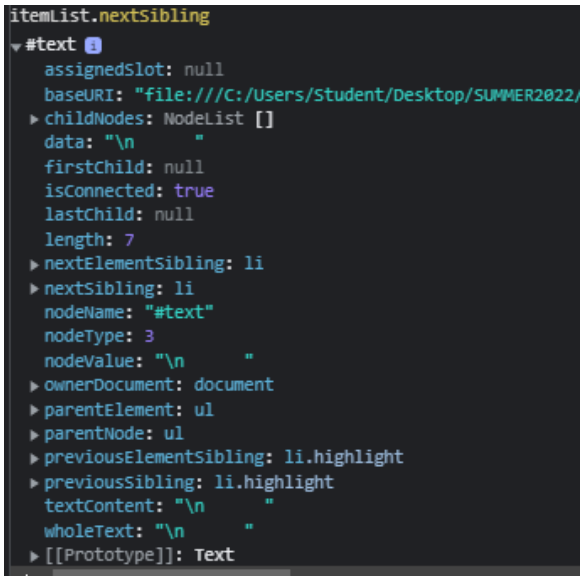
```
> const itemList= document.querySelector('li')
< undefined
> itemList
< ▼<li class="highlight">
    ::marker
    "Apple"
  </li>
> itemList.parentElement
< ▶<ul>…</ul>
> itemList.parentElement.parentElement
< ▶<body>…</body>
> itemList.parentElement.parentElement.parentElement
<   <html>
    ▶<head>…</head>
    ▶<body>…</body>
    </html>
```

We can also inspect the child elements of the list items using the 'children' and 'childElementCount' properties.

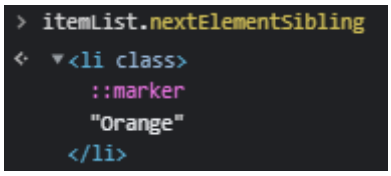
```
> itemList.childElementCount
< 0
> itemList.children
< ▼HTMLCollection [] i
    length: 0
  ▶[[Prototype]]: HTMLCollection
```

Sibling property returns information of the same type adjacent to the element.

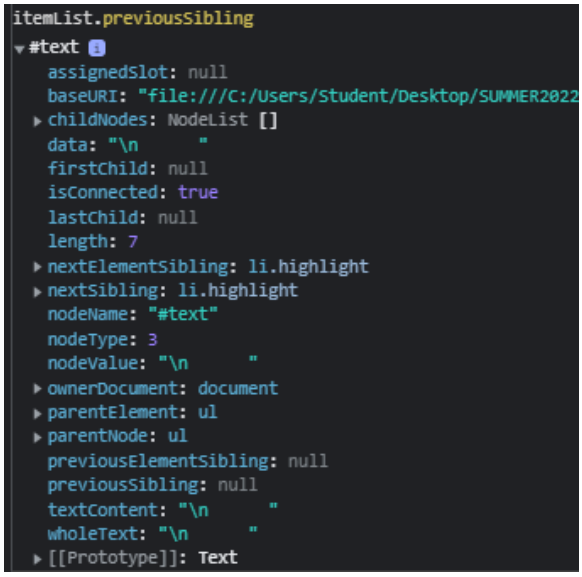
Property **.nextSibling** returns the node text of the next adjacent sibling element:



Property .nextElementSibling returns the element the content of the next adjacent element of the same type:



previousSibling property



previousElementSiblings

```
> const ulList = document.querySelector('ul')
< undefined
> ulList
< ▶<ul>…</ul>
> ulList.previousElementSibling
<   <p>Fruit Shopping List</p>
```

4.5. Creating Elements and Text Nodes

It can be easy to add HTML to a page using innerHTML, as you have already seen, but sometimes you need to actually create an element, put some text in it, and add it to the page. To add a new element to the HTML DOM, you must create the element (element node) first, and then append it to an existing element.



Example 12) create an image node and append it to an existing element in the DOM

```
// create a content
const newImg = document.createElement('img')
newImg.src = 'iceland.jpg'
// append newImg to the end of body element
document.body.appendChild(newImg)
newImg.style.width = '20%';
```

Example 13) create a paragraph and append it after the element

```
<div>
  <p>A paragraph</p>
</div>
```

```
<script type="text/javascript">
// step 1: create a new paragraph
   let myPara = document.createElement('p');
// step 2: give it content
   let mySentence = document.createTextNode('This is a text in a new
paragraph');
// step 3: add it to the DOM
   myPara.appendChild(mySentence);
// step 3: find the position where the new element will be added
   document.querySelector('div').appendChild(myPara);
</script>
```

Example 14) Create a subtitle using <h2> and append it next to an element with class name **.n1**

```
 <div id="special">
    <p >First Name</p>
    <p>Last Name</p>
    <p class="n1">Full Name</p>
 </div>

<script>
      const newSubtitle = document.createElement('h2')
      newSubtitle.innerHTML ='Appending elements to the DOM';
      document.querySelector('.n1).appendChild(newSubtitle)
</script>
```

Example 15) create 20 buttons and add them inside with id="container"

```
<div id="container"> </div>

<script>
  for (let button = 1; button<=20; button++){
    let b = document.createElement('button')
    b.innerHTML =`Button ${button}`
    document.body.appendChild(b)
}
</script>
```

We also add element to the end of an element by using the method **.append().** We can add element to the beginning to an element by using **prepend()**

Note: IE does not support `append()` and `prepend()`

Example 16) append a message and an image stores in variable `newImg` from Example 16 to the `<div>` element with `id="special"`

```
<div id="special">
      <p >First Name</p>
      <p>Last Name</p>
      <p class="n1">Full Name</p>
    </div>

<script>
    document.querySelector('#special').append('HELLO THERE!',newImg)
</script>
```

Example 17) using example 16, append a paragraph to the beginning of `<div id="special">`

```
document.querySelector('#special').prepend('HELLO THERE!',newImg)
```

We can also append element to the adjacent elements, for example in the middle of elements. This can be done using an `.insertAdjacentElement()`. For this, we need to specify a position (`beforebegin, afterbegin, beforeend, afterend`) of the appended element.

Example 18)

```
<div id="special">
      <p >First Name</p>
      <p>Last Name</p>
      <p class="n1">Full Name</p>
</div>

<script>
    const subtitle = document.createElement('h2')
    subtitle.append("THINGS TO DO IN NYC!")
    let nextSubtitle = document.querySelector('#special')
    nextSubtitle.insertAdjacentElement('afterend',subtitle)
</script>
```

4.6. Removing Elements

There are two ways to remove a child from a DOM: `removeChild()` and `remove()`

The original method is `removeChild()`, it has all browser support but it is more complicated to use it. It does not remove the selected element; it removes the child of the selected element.

Example 19)

```
<div>
  <p class = 'p1'>PARAGRAPH 1</p>
  <p>PARAGRAPH 2</p>
</div>

<script type="text/javascript">
    let myDiv = document.querySelector('div');
    myDiv.removeChild(myDiv.children[0]);
</script>
```

Or we can write

```
<script type="text/javascript">
    let par1 = document.querySelector('.p1');
    par1.parentElement.removeChild(par1);
</script>
```

A new method to remove an element in the DOM is called **`remove()`**

Example 20) using the previous example, remove <p class = 'p1'> using the remove() method

```
let par1 = document.querySelector('.p1');
par1.remove()
```

4.7. Capturing Events with JavaScript

HTML events are **"things"** that happen to HTML elements. When JavaScript is used in HTML pages, JavaScript can **"react"** on these events. An HTML event can be something the browser does, or something a user does.

Here are some examples of HTML events:

- An HTML web page has finished loading.
- An HTML input field was changed.
- An HTML button was clicked.

Often, when events happen, you may want to do something.

JavaScript lets you execute code when events are detected.

HTML allows event handler attributes, with JavaScript code, to be added to HTML elements.

When the user interacts with the HTML on a web page, there are three steps involved in getting it to trigger some JS code. Together, there is three main steps how JS handle events:



4.7.1. Ways to bind an event to an element

HTML event handler attributes

Early versions of HTML included a set of attributes that could respond to events on the element they were added to. This approach is now considered bad practice; however, you need to be aware of it because you may see it at older code.

Inline events

Inline events happen when we place the event inline in the open tag of the element in the HTML code. It is not recommended but it exist and it is possible

Example 21) apply inline event to a button that when it is clicked it opens an alert window.

```
<button onclick="alert('You clicked!')">CLICK HERE</button>
```

Traditional DOM event handlers

All modern browsers understand this way of creating an event handler, but you can only attach one function to each event handler.

Syntax:

```
Element.onevent = functionName;
```

Where:

- element is the DOM element to be targeted.
- **onevent** is the event bound to node(s) added by word "on".
- functionName is the name of function to call. This function doesn't have parentheses.

Example 22) use even handler to a button that displays an alert message when is clicked.

```
<button>PRESS ME!</button>

<script type="text/javascript">
    let btn = document.querySelector('button');
    btn.onclick = function(){ alert("Hi There!");}
     // anonymous function- good for only one run
</script>
```

We can also write a function separately and call the function in the event handler line. Naming the function separately can be used to different event handler:

```
//named function: use for multiple runs
function clickedBtn(){
   alert("Hi There!");
}
btn.onclick = touch;
```

The most common HTML event handlers are:

List of Event Handler	
Event handler	**Description**
onchange	An HTML element has been changed
onclick	Invoked when a clickable element is clicked or tapped.
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page
onhover	Triggered when the cursor hovers over an element.
onsubmit	Fired when a form is submitted.
onfocus	Activated when an element receives focus.
onblur	Fired when an element loses focus.

Table 1 - List of event handler

Table 1 provides a comprehensive overview of common event handlers utilized in web development, outlining their respective functions and triggering conditions.

Example 23) create a link that when a mouse goes over it or when it is clicked, a message will display in the console.

```
<a href="https://www.qcc.cuny.edu/" target="_blank"
id='qccLink'>QCC</a>

<script type="text/javascript">
      const aLink = document.querySelector('#qccLink');
      aLink.onclick = function(){
      console.log('I WAS CLICKED!')
      }
      function testing(){
      console.log('HOVER!')
      }
      aLink.onmouseover = testing;
  </script>
```

We can also add an event directly to an element.

Example 24) create an event to <h1 class="title"> that when a mouse leaves the element, it will display a message in the console.

```
<h1 class="title">Events</h1>

<script>
  document.querySelector('.title').onmouseout =
function(){alert('MOUSEOUT!')}
</script>
```

You can also use an arrow function as:

```
document.querySelector('.title').onmouseout =() =>{alert('MOUSEOUT!')}
```

4.7.2. Event listeners

Event listeners are a more recent approach to handling events. They can deal with more than one function at a time, but they are not supported in older browser.

The **`addEventListener()`** method takes three parameters:

- Event you want to listen for
- The code that you want it to run when the event fires
- A Boolean indicating how events flow. Usually is set to **false**.

Syntax:

```
Element.addEventListener('event', functionName, boolean);
```

Where:

- “element” is the DOM element node to be targeted.
- “addEventListener” method.
- “event” is the name of the event that will be bound to the DOM node. The event name is enclosed in quotation marks.

The browser can trigger many different types of events on the DOM(Document Object Model). The full list of all DOM event types is located developer.mozilla.org website [Event reference | MDN - https://developer.mozilla.org/en-US/docs/Web/Events]

Here are some of the most common event types and event names:

- Mouse Events: `click, dblclick, mousedown, mouseup, contextmenu, mouseout, mousewheel, mouseover.`
- Touch Events: `touchstart, touchend, touchmove, touchcancel.`
- Keyboard Events: `keydown, keyup, keypress.`
- Form Events: `focus, blur, change, submit.`
- Window Events: `resize, scroll, load, unload, hashchange.`

Touch events are triggered by touch-enabled devices such as smartphones, tablets, and touch-screen laptops. Mouse events are triggered on most of all browsers and devices. The Mouse event interface represents events that occur due to the user interacting with a pointing device.

The `click` event occurs when the user clicks on an element. The `dblclick` event occurs when the user double-clicks on an element.

`mousedown` and `mouseup` events occur when pointing device button is pressed/released on an element. `mouseout` event occurs when a pointing device is moved off the element that has the listener attached.

Keyboard events(`keydown, keyup, keypress`):

- `keydown`: Any key is pressed.
- `keyup:` Any key is released.
- `keypress`: Any key (except shift, Fn, or capslock) is in the pressed position(fired continuously.)

Form Events(`focus, blur, change, submit`):

- `focus:` An element that has received focus.
- `blur:` An element that has lost focus.
- `change`: Event that is fired for input, select, and textarea elements when an alteration to the element's value is done by the user.
- `submit`: The submit button is pressed.

Window Events (`resize, scroll, load, unload, hashchange`):

- `resize`: This event fires when the document view(window) has been resized.
- `scroll:` Event fires when the document view or an element has been scrolled.
- `load/unload`: The load event is fired when the whole page has loaded, including all resources such as css and images. Unload is when the document or child resource is being unloaded.
- `hashchange:` This event is fired when the identifier of the URL has changed(the part of the URL that begins with a # symbol).

Some other common DOM events that are used are:

- `error`: A resource has failed to load.
- `abort`: The loading fo a resource was aborted.
- `online:` The browser has gained access to the network.
- `animationstart:` This event fires when a CSS animation has started.

Event listeners are what make JavaScript dynamic and fun. It allows the developer to be creative and experiment with their application to make it interesting and expressive. There are all kinds of keyword events that just about allow the user to do anything to interact with the browser. Knowing more event listener types will allow you(the developer) to become a JavaScript wizard! [3]

4.7.3. Mouse event

Mouse events are event that happen with the features of a mouse. Some of the mouse events are: `click, dblclick, mousedown, mouseup, contextmenu, mouseout, mousewheel, mouseover`

[3] *Add Event Listener DOM Event Types,* retrieved from https://dbchung3.medium.com/add-event-listener-dom-event-types-6c10a844c9d8, 7/19/2020

Click event

The click event occurs when the user clicks on an element. dblclick event occurs when the user double-clicks on an element.

Example 25) Use `addEventListener()` method that when a button is clicked, it will display an alert window.

```
<button id='btn5'>PRESS ME!</button>

<script type="text/javascript">
    let btn5 = document.querySelector('btn5');
      btn.addEventListener('click', function(){
        alert("Hi There!");}
);
```

Example 26) create an event that when a button is clicked, it will randomly select a background color a container.

HTML code

```
<div class="color">
     <button id='btnColor'>CHANGE COLOR</button><p
class="displayRGB"></p>
</div>
```

JavaScript code

```
//step 1: collect and save the elements
let colorcontainer = document.querySelector(".colorcontainer")
let btncolor = document.querySelector(".btncolor")

//step 2: bound an event to the 'btncolor'
btncolor.addEventListener("click", function(){
    colorcontainer.style.backgroundColor = randomcolor()
})

// change the background color of element 'colorcontainer'
let randomcolor = function(){
    let redvalue = Math.floor(Math.random()*255)
    let greenvalue = Math.floor(Math.random()*255)
    let bluevalue = Math.floor(Math.random()*255)
    return `rgb(${redvalue},${greenvalue},${bluevalue})`
}
```

mouseove *and* mouseout *event*

The **mouseover** event takes place when the pointer of the mouse comes over an element. On the contrary, the **mouseout** event occurs when it leaves.

Example 27) Use **`addEventListener()`** method to add a **`mouseover`** and **`click`** event to the same button

```
  <button id="btn2">Try it</button>
  <p class="display">MESSAGE WILL ADD HERE<br></p>

  <script>
       let button2 = document.querySelector("#btn2");
       button2.addEventListener("mouseover", hover1);
       button2.addEventListener("click", alert1);

       function hover1() {
         document.querySelector(".display").innerHTML += "Moused
     over!<br>";
       }

       function alert1() {
         document.querySelector(".display").innerHTML +=
     "Clicked!<br>";
       }
  </script>
```

Example 28) Use addEventListener() method to add two click events to the same button

```
<button id="btn6">CLICK ME!</button>

<script>
    let btn6 = document.querySelector("#btn6");
    btn6.addEventListener("mouseout", click1);
    btn6.addEventListener("dblclick", click2);

    function click1() {
      console.log('BUTTON 6 = mouseout');
    }

    function click2() {
      alert ("It's me again! Double click");
    }
</script>
```

Alternate way to write the script is:

```
<script>
    let button1 = document.querySelector("#btn");
    button1.addEventListener("mouseout", function(){
        console.log('BUTTON 6 = mouseout');
    });
    button1.addEventListener("dblclick", function(){
        alert ("It's me again! Double click");
    });
</script>
```

Example 29) using example 7, create 12 elements that when a mouse moves out from a its background color changes.

```
<section style="display:inline-block">
    <div class="divColor"></div> <div class="divColor"></div>
    <div class="divColor"></div> <div class="divColor"></div>
    <div class="divColor"></div> <div class="divColor"></div>
    <div class="divColor"></div> <div class="divColor"></div>
</section>

<script type="text/javascript">
   const  randomRGB = function(){
        let randomR = Math.floor(Math.random()*255)
        let randomG = Math.floor(Math.random()*255)
        let randomB = Math.floor(Math.random()*255)
        return `rgb(${randomR}, ${randomG}, ${randomB})`
      }

    const divColor = document.querySelectorAll('.divColor');
      for(let divEach of divColor){
         divEach.addEventListener('mouseout', function(){
          divEach.style.backgroundColor = randomRGB();
        })
      }
</script>
```

4.7.4. Keyboard events

Keyboard event happens when interacting with keys in the keyword. Keyboard events are:

- **keydown**: Any key is pressed.
- **keyup**: Any key is released.
- **keypress**: Any key (except shift, Fn, or capslock) is in the pressed position(fired continuously.)

Example 30) create a text input that when any key is pressed from the keyboard into the text field, an alert window will pop

```
<input type="text" class="inputTxt">

<script type="text/javascript">
  const inputTxt = document.querySelector('.inputTxt')
  inputTxt.addEventListener('keydown', function(){
        alert('KEYDOWN!')
      })
</script>
```

We can also create a parameter for the event function. The parameter e is by default an event parameter and it has different properties. Some of those properties are key and code. key returns the key that was pressed, and code returns the key code. More examples are posted at event object.

4.7.5. DOM objects

The event object

There is an event object that can be passed into event handlers, and sometimes you need to access it specifically. When an event occurs, the event object tells the information about the event and the element it happened upon. In JavaScript, the event object is a fundamental part of handling events triggered by user interaction or system events in web development. When an event occurs (like a mouse click, key press, or window resize), JavaScript creates an event object containing information about that event. Developers can then use this object to access details about the event and respond accordingly.

Every time an event fires, the event object contains helpful data about the event, such as:

- Which element the event happened on
- Which key was pressed for a keypress event
- What part of the viewport the user clicked for a click event.

The event object is passed to any function that is the event handler or listener. If you need to pass arguments to a named function, the event object will first be passed to the anonymous wrapper function, it happens automatically. Then it must specify it as a parameter of the named function.

When the event object is passed into a function, it is often given the parameter name e, for event. It is a widely used shorthand. However, some programmer uses the parameter name e to refer to error object.

Some of the event objects properties and methods are:

Event object property description		
Event object Property	**IE5 to IE8 equivalent**	**Purpose**
target	srcElement	References the element that triggered the event (most specific element interacted with)
type	type	Indicates the type of event that was fired, for example, ‘click’, ‘scroll’, etc
cancelable	Not supported	Whether you can cancel the default behavior of an element
Method	**IE5 to IE8 equivalent**	**Purpose**
preventDefault()	returnValue	Cancel default behavior associated with the event (if it can be canceled)
stopPropagation()	cancelBubble	Stops the event from bubbling or propagating to parent elements.
event.clientX event.clienY		Provide the horizontal and vertical coordinates of the mouse pointer when the event occurred (for mouse-related event)

Table 2 - event object properties

One application of event object is with the keyboard event. We can also check which key was pressed by given an argument to the anonymous function:

- `which` method shows the ASCII code of the selected key.
- `key` method shows the key character that you have pressed.
- `keyCode` method returns the key code of the pressed key.

Example 31) The program will prompt the ASCII code and the key that was pressed

```
// Add an event listener for keydown event
document.addEventListener("keydown", function (event) {
   // prompt the ASCII code of the pressed key
   console.log(`ASCII code of key pressed: ${event.keyCode}`);
   // prompt the pressed key
   console.log(`Key '${event.key}' was pressed`);
   // prompt the ASCII code of the pressed key using 'which'
   console.log(`ASCII code of key pressed: '${event.which}' `);
});
```

Example 32) Use event object to change the button's background color and message when it is clicked.

When the browser runs, it displays a capsule-shaped button with a dark gray background and the label "PRESS ME."



When the button is clicked, its background color changes to red, and the text updates to "Key was pressed!".



HTML code

```
<button class="btnpressme">PRESS ME</button>
```

CSS code

```
.btnpressme{
   padding: 0.5em 2em; color: whitesmoke;
   border-radius: 12px; background-color: rgb(50, 50, 50);
}
.btnactive{
   background-color: crimson; font-weight: bolder;
}
```

JavaScript code without event object:

```
// get the button
let btnpressme = document.querySelector("button.btnpressme")
// add event listener was button is clicked
btnpressme.addEventListener("click", function(){
     //toggle the text content of a button
     if (btnpressme.textContent ==="PRESS ME"){
           btnpressme.textContent ="Key was pressed!"
     }
      else{
            btnpressme.textContent ="PRESS ME"
     }
     //toggle class in a button
     btnpressme.classList.toggle("btnactive")
})
```

JavaScript code with event object

```
btnpressme.addEventListener("click", function(e){
//toggle the text content of a button
    if (e.target.textContent ==="PRESS ME"){
          e.target.textContent ="Key was pressed!"
    }
    else{
          e.target.textContent ="PRESS ME"
    }
    //toggle class in a button
          e.target.classList.toggle("btnactive")
})
```

Event Delegation

Event delegation is applied when we have a lot element s handled in a similar way, then instead of assigning a handler to each of them – we put a single handler on their common ancestor.

In the handler, we use **`event.target()`** to see where the event actually happened and handle it. 4

In JavaScript, when referring to the "target" properties, we're usually talking about the properties of the target element that triggered an event. These properties are accessed through the target property of the event object.

Here are some common properties you can access on the target object:

- **`target.tagName:`** Returns the tag name of the element as a string (e.g., "DIV", "SPAN", "INPUT").
- **`target.id:`** Returns the id attribute of the element as a string.
- **`target.className:`** Returns the class attribute of the element as a string.
- **`target.value`**: For form elements (like <input>), returns the current value of the input field.
- **`target.href:`** For anchor elements (<a>), returns the href attribute, which contains the URL the anchor points to.
- **`target.dataset:`** Returns a DOMStringMap object containing all the data attributes of the element.
- **`target.parentElement:`** Returns the parent element of the target element.
- **`target.childNodes:`** Returns a live collection of child nodes of the target element.

These are just a few examples, and there are many more properties available depending on the type of element and the event being handled. You can access these properties to gather information about the target element or to manipulate it in response to the event.

[4] *Event Delegation,* JavaScript.info, https://javascript.info/event-delegation, retrieved on July 2022

Example 33) use event delegation to remove an item in the list when it is clicked.

HTML code

```
<ul id="list">
    <li>Apple</li>
    <li>Orange</li>
    <li>Kiwi</li>
</ul>
```

CSS code

```
 #list li{
     width: 200px;
     padding: 1em 2em;
     border: lightblue 1px solid;
     list-style: none;
 }
 #list li:hover{
     background-color: crimson;
}
```

JavaScript code

```
 // Get ul element with id list
 const list = document.querySelector("ul#list");

 // Add event listener to the parent element
 list.addEventListener("click", function (event) {
      // Check if the clicked element is an <li> element
     // usually the return tagName is all upper case, therefore we
need to change the tag name to lower case using the method ‘toLoa
      if (event.target.tagName.toLowerCase() === "li") {
          // Remove the clicked <li> element
            event.target.remove();
      }
 });
```

In JavaScript, the tagName property of an element returns the tag name of the element in uppercase letters. However, HTML tag names are case-insensitive, meaning that "LI", "li", "Li", and "lI" all refer to the same element type: a list item (<li>).

To ensure compatibility and consistency, it's a good practice to perform case-insensitive checks when comparing tag names in JavaScript. By converting both the tagName property and the comparison string to lowercase using the toLowerCase() method, you can ensure that the comparison works regardless of the case of the tag name.

preventDefault()

The preventDefault() method cancels the event if it is cancelable, meaning that the default action that belongs to the event will not occur.

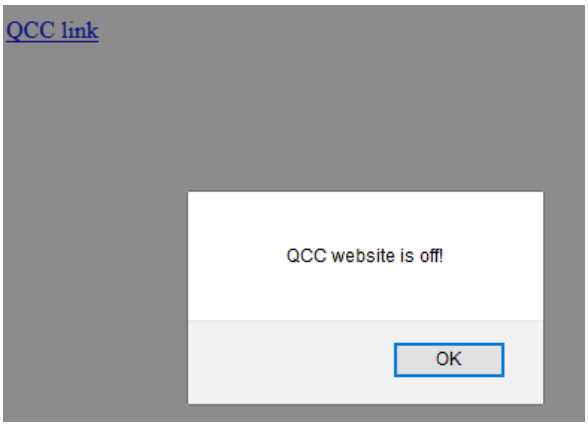
For example, this can be useful when:

- Clicking on a "Submit" button, prevent it from submitting a form
- Clicking on a link, prevent the link from following the URL

Example 44) Use preventDefault() method to prevent the link to QCC to open when is clicked

```
<a href=www.qcc.cuny.edu class>QCC link</a>
<script type="text/javascript">
    let link = document.querySelector('a');
    link.addEventListener('click',function(e){
      e.preventDefault();
      alert('QCC website is off!');
    });
</script>
```

The output in the browser will indicate that the <a> element does not have a functioning link, as depicted in Figure 2.



If we remove e.preventDefault(); line from the code, when we click on the QCC link, the alert message will appear and the load www.qcc.cuny.edu website.

Event Bubbling

When an event happens to an element, it first runs the handlers on it, then on its parent, then all the way up on other ancestors.

Example 35) Use three nesting elements with click event on it. When an element is clicked, it opens an alert window.

```
<section class="section"> SECTION
   <p class="pClick">PARAGRAPH
      <button class="button">CLICK BUTTON</button>
   </p>
</section>

<script>

const s1 = document.querySelector('.section')
const p1 = document.querySelector('.pClick')
const b1 = document.querySelector('.button')
s1.addEventListener('click', function(){
      alert('Section clicked!')
})
p1.addEventListener('click', function(){
      alert('Paragraph clicked!')
})
b1.addEventListener('click', function(e){
   e.stopPropagation();
   alert('Button clicked!')
})

</script>
```

In order to stop bubbling, there is an event method called stopPropagation() stopPropagation() will run the current event and prevent propagation to the any parent or sibling events.

4.7.6. Window event

Window events happen when user interact with the browser window. Window events include:

- `load:` when the browser finishes the loading of the page.
- `unload:` when the visitor leaves the current webpage, the browser unloads it.
- `resize:` when the visitor resizes the window of the browser.
- `scroll:` Event fires when the document view or an element has been scrolled.

Scroll event

When you scroll a document or an element, the scroll events fire. Scroll events on the window object to handle the scroll of the whole webpage.

To register a scroll event handler, you call the addEventListener() method on the target element, like this:

Syntax:

```
targetElement.addEventListener('scroll', (event) => {
          // handle the scroll event
});
```

or assign an event handler to the onscroll property of the target element:

```
targetElement.onscroll = (event) => {
          // handle the scroll event
};
```

Scroll property

The DOM window object has a scrollBy() method that allows the window to be scrolled horizontally and vertically when content overflows the window in either orientation. This method requires two arguments to specify the number of pixels to shift along the X and Y axes.

When content overflows the window vertically, a scroll bar appears along the right edge of the browser window. The scrollBy() method will scroll by the number of pixels specified as its first argument – or unit it reaches the extreme of the content.

Example 36) create a button that will scroll 100px to the right of x-axis

HTML code

```
<div id="content">
  <h1>Scrollable Content</h1>
  <p>This content is wider than the window, allowing for horizontal
scrolling.</p>
</div>

<button class="btnscrollby">Scroll Right</button>
```

CSS code

```
#content {
       width: 2000px; /* To create a horizontal scrollable area */
       background-color: lightblue;
       padding: 20px;
  }
```

JavaScript code

```
let btnscrollby = document.querySelector("button.btnscrollby")
btnscrollby.addEventListener("click", scrollContent)

function scrollContent() {
// Scrolls the window horizontally by 100 pixels and 0 pixels in the
‘y’ axis
      window.scrollBy(100, 0);
}
```

scrollBy() also comes with key properties such as:

- **`x:`** Specifies the number of pixels to scroll horizontally. A positive value scrolls to the right, while a negative value scrolls to the left. This key is also called **`left`**
- **`y:`** Specifies the number of pixels to scroll vertically. A positive value scrolls down, while a negative value scrolls up. This key is also called **`top`**
- **`behavior:`** Specifies the scrolling behavior. This property determines whether the scrolling should be smooth or instant.
    - **`"auto":`** Indicates that the scrolling should happen instantly, without animation.
    - **`"smooth":`** Indicates that the scrolling should animate smoothly.

Example 37) here's a simple application that demonstrates the use of scrollBy() to create a horizontal scrolling effect for a gallery of images:

HTML code

```
<section class="maincontainer">
   <div class="gallerycontainer">
      <img class="galleryimage" src="images/image1.jpg" alt="Image 1">
      <img class="galleryimage" src="images/image2.jpg" alt="Image 2">
      <img class="galleryimage" src="images/image3.jpg" alt="Image 3">
      <img class="galleryimage" src="images/image4.jpg" alt="Image 4">
   </div>
   <button class="btnright">&#x276F;</button>
   <button class="btnleft"> &#x276E;</button>
 </section>
```

CSS code

```
.gallerycontainer, .maincontainer{
     width: 1000px;
     margin: auto;
}
.maincontainer {
     position: relative;
     z-index: 0;
}
.gallery-container {
     overflow-x: scroll; /* Enable horizontal scrolling */
      white-space: nowrap; /* Ensure all images stay on the same line
*/
      position: relative;
      z-index: 1;
}
.gallery-image {
      display: inline-block;
      width: 502px;
      margin: -2px; /* Space between images */
}
.btnleft, .btnright{
      position: absolute;
      z-index: 2;
      top: 40%;

}
```

```
.btnright {
      right: 1%;
}
.btnleft {
      left: 1%;
}
```

JavaScript code

```
/*getting the left and right buttons*/
const btnright = document.querySelector("button.btnright")
const btnleft = document.querySelector("button.btnleft")

/*define a function to move scroll the photo gallery by the number of
pixels*/
function scrollGallery(pixels) {
     document.querySelector('div.gallerycontainer').scrollBy({
          left: pixels,
          behavior: "smooth"
     })
}

/*add event to each button when is clicked*/

btnright.addEventListener("click", function () {
     scrollGallery(500)
})
btnleft.addEventListener("click", function () {
     scrollGallery(-500)
})
```

The other way to set this JS code is:

```
let content = document.querySelector("#content")
let btnscrollright = document.querySelector(".btnscrollright")
btnscrollright.addEventListener("click",function(){
    window.scrollBy(100,0)
})
/* slice photo gallery */
let btnleft = document.querySelector(".btnleft")
let btnright = document.querySelector(".btnright")
let gallerycontainer = document.querySelector(".gallerycontainer")

btnleft.addEventListener("click", function(){
    gallerycontainer.scrollBy({
```

```
        left: -500,
        behavior: "smooth"
    })
})

btnright.addEventListener("click", function(){
    gallerycontainer.scrollBy({
        left: 500,
        behavior: "smooth"
    })
})
```

The scrollTo() method in JavaScript is used to scroll the document to a specified position. It's commonly used to smoothly scroll to a particular section of a webpage or to a specific coordinate on the page. scrollTo() method that accepts two arguments specifying X and Y coordinates that the top-left corner of the window should scroll to when content overflows the window horizontally and vertically. This can be used to shift away from the default X=0, Y=0 coordinates to a specified alternative position.

Example 38) In this example:

- We have a simple webpage with a link ("Scroll to Top") positioned at the bottom right corner of the viewport.
- When the link is clicked, the JavaScript code attached to it prevents the default behavior of jumping to the top of the page.
- Instead, it scrolls smoothly to the top of the document using the window.scrollTo() method. The behavior: 'smooth' option ensures the scrolling animation is smooth rather than instantaneous.

HTML code

```
<div style="height: 2000px;">
        <p>Scroll down...</p>
        <a href="#" id="scrollToTop">Scroll to Top</a>
</div>
```

CSS code

```
.section {
    height: 500px;
    margin-bottom: 20px;
    border: 1px solid #ccc;
    padding: 20px;
}
.scrollcontainer{
    height: 3000px;
    background-color: #AAA;
}
```

JavaScript code

```
 // Get reference to the scroll to top link
const scrollToTopLink = document.querySelector('#scrollToTop');
// Add click event listener to the link
scrollToTopLink.addEventListener('click', function (event) {
     // Prevent the default anchor behavior
     event.preventDefault();
     // Scroll to the top of the document (0,0 coordinates)
      window.scrollTo({
           top: 0,
           behavior: "smooth"
       })
});
```

The DOM window object has a scrollY property that stores the number of pixels by which the window is scrolled vertically. This denotes the position of the thumb along the scrollbar at the right of the window, relative to its top corner. The window.scrollY property is an alias for an older property named window.pageYOffset that still exists in the DOM, and can be used instead of window.scrollY for backward compatibility.

Example 38) Show the pixels from the top of the window browser when scroll down.

HTML code

```
<p class="display9"></p>
```

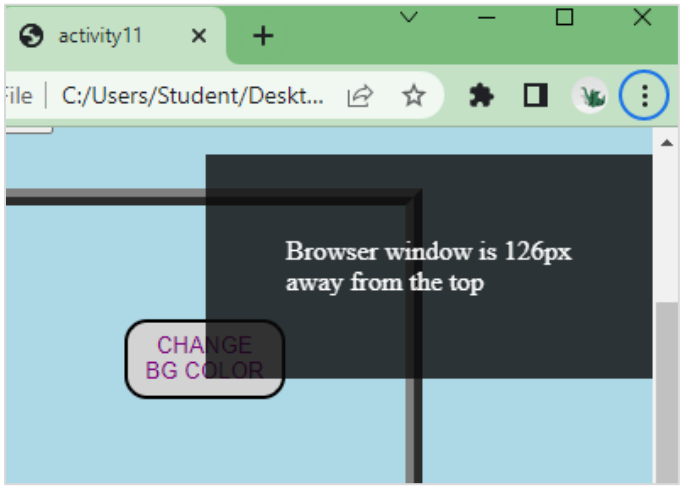
CSS code

```
.display9{
      position: fixed;  right: 0%; top: 50%;
      padding: 3em;
      background-color: rgba(10,10,10,0.8);
      width: 50%;
      color: white;
}
```

JavaScript code

```
// get display paragraph
const display9 = document.querySelector('.display9')
// window scroll
window.addEventListener('scroll', function () {
    let pxTop = window.scrollY // alternative way ->
window.pageYOffset;
    display9.innerHTML = `Browser window is ${pxTop}px away from the
top `
});
```

In the browser, when we scroll, it displays the pixel height of the box.



Example 37) Using the window property window.pageYOffset; create a link that appears when the window is scrolled 120px away from window, and disappears when the window is closer to the top by less than 120px.

HTML code

```
<div class="toTop">
        <a href="#">&#x270C;<br> Top </a>
</div>
```

CSS code

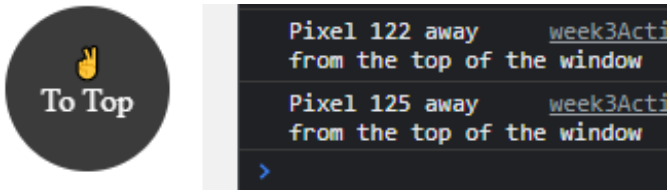
```
html { scroll-behavior: smooth; }

.toTop {
      position: fixed;
      bottom: 10%;
      right: 10%;
      width: 60px;
      height: 60px;
      border-radius: 50%;
      background-color: rgba(20, 20, 20, 0.9);
      text-align: center;
      display: none;
}
.toTop a {
      text-decoration: none;
      color: whitesmoke;
}
.toTop:hover {
      box-shadow: 0px 0px 5px 2px black;
      opacity: 0.8;
}
```

JavaScript code

```
// get element
const toTop = document.querySelector('.toTop')
// window scroll
window.addEventListener('scroll', function () {
     let pxTop = window.scrollY;
     // display the division when the window is scrolled > 80px
     if (pxTop > 80) { toTop.style.display = 'block'; }
     // hide the division when the window is scrolled <= 80px
     else { toTop.style.display = 'none'; }
})
```

The result in internet browser will look as the following:



Window screen object

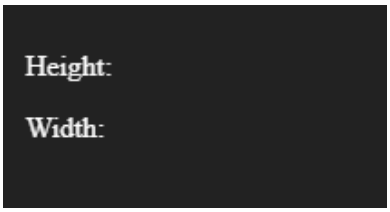
The top-level DOM window object has a screen child object that provides properties describing the user's monitor resolution in the pixel measurement. Overall screen dimensions can be found in the window.screen.width and window.screen.height properties.

```
console.log(`Screen width ${window.screen.width}`)
console.log(`Screen width ${window.screen.height}`)
```

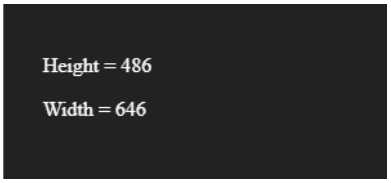
Window resizing

The resize event fires when the document view (window) has been resized. In some earlier browsers it was possible to register resize event handlers on any HTML element. It is still possible to set onresize attributes or use addEventListener() to set a handler on any element. However, resize events are only fired on the window object (i.e. returned by document.defaultView). Only handlers registered on the window object will receive resize events.

Example 38) Use the resize event to display the height and width of the window. The internet browser should look as follows:



And when the window is scrolled down, the division will prompt the distance away from the top in pixels.



HTML code

```
<div class="resizeInfo">
  <p class="rHeight">Height: </p>
  <p class="rWidth">Width: </p>
</div>
```

CSS code

```
*{box-sizing:border-box;}
.resizeInfo{
   padding: 5%;
   width: 50%;
   position: sticky; top: 0px;
   background-color: rgba(10,10,10,0.9);
   color: white;
}
```

JavaScript code

```
let windowH = document.querySelector('.rHeight');
let windowL = document.querySelector('.rWidth');

window.addEventListener('resize', function(){
  let readH = window.innerHeight;
  let readW = window.innerWidth;
  windowH.innerHTML = `Height = ${readH}`;
  windowL.innerHTML = `Width = ${readW}`;
});
```

4.7.7. Form events

Form events in JavaScript are events that occur on HTML form elements, such as <input>, <select>, <textarea>, and <button>. These events can be captured and handled by JavaScript to perform actions based on user interactions with the form. Some common form events include:

- **`submit`**: This event is triggered when the user submits a form, typically by clicking a submit button or pressing Enter while in a form field.
- **`reset:`** This event is fired when the user clicks a reset button within a form, resetting all form fields to their initial values.
- **`input:`** This event is triggered when the value of an **`<input>`**, **`<select>`**, or **`<textarea>`** element changes. It is often used to provide real-time validation or to update UI elements as the user types.
- **`change:`** This event is similar to the input event but is typically fired when the value of an input element changes, and the element loses focus. It is commonly used for validation or to trigger actions when the user has finished entering data.
- **`focus:`** This event occurs when an element within the form gains focus, such as when a user clicks on a form field or navigates to it using the keyboard.
- **`blur:`** This event is fired when an element within the form loses focus, such as when the user clicks outside of a form field or tabs away from it.
- **`keydown, keyup, keypress:`** These events are triggered when a key is pressed down, released, or both, respectively, while the focus is on an input element. They are commonly used for tasks like capturing keyboard input or implementing keyboard shortcuts within forms.

These are some of the most commonly used form events in JavaScript. By handling these events, you can create interactive and dynamic forms that respond to user input in various ways.

Submit event

The submit event in JavaScript occurs when a form is submitted, either by clicking a submit button or by pressing Enter while in a form field. This event is commonly used to perform actions such as form validation, data processing, or sending form data to a server for further processing (via AJAX or a traditional form submission).

Example 39) In this example:

- We have an HTML form with an input field for the username and a submit button.
- We use JavaScript to add an event listener for the submit event on the form element.
- Inside the event listener function, we prevent the default form submission behavior using event.preventDefault().
- We perform form validation by checking if the username field is empty. If it's empty, we display an alert and return early from the function.
- If the validation passes, we can access the form data (in this case, the username) and perform further actions, such as sending the data to a server or logging it to the console.

Remember that preventing the default form submission behavior (event.preventDefault()) is important if you want to handle form submission using JavaScript. This allows you to process the form data and perform any necessary actions before submitting it to the server.

HTML code

```
<form id="myForm">
   <label for="username">Username:</label>
   <input type="text" id="username" name="username" placeholder="Type
username...">
   <button type="submit" class="submitbtn">Submit</button>
</form>

!-- Display message after form is submitted -->
<div class="greeting"><p>Welcome to form event <span></span></p></div>
```

CSS code

```
#myForm{
   font-family:Arial, Helvetica, sans-serif;
   background-color: azure;
   width: 30%;
   margin: 5em auto;
   padding: 3em 5em;
}
```

```
.submitbtn{
   background-color: crimson;
   padding: 0.5em 2em;
   border-radius: 5px;
   border: none;
   color:white;
   cursor: pointer;
}
.submitbtn:hover{
   background-color: #333;
}


.greeting {
   font-size: 1.5em;
   padding: 1em;
   display: none;
   width: 30%;
   margin:auto;
   text-align: center;
}
.greeting p span {
   font-family: "algerian";
}
```

JavaScript code

```
// Get reference to the form element
const myform = document.querySelector('#myForm');
// Get the greeting <div>
const greeting = document.querySelector('.greeting');
const greetingname = document.querySelector('.greeting p span')

// Add event listener for the submit event
myform.addEventListener('submit', function (event) {
     // Prevent the default form submission behavior
     event.preventDefault();

     // Perform form validation or other actions here
     const usernameInput = document.querySelector('#username');
     const username = usernameInput.value;

     if (username.trim() === '') {
           alert('Please enter a username.');
           return; // Stop further execution
      }
```

```
        /* If validation passes, you can submit the form data to a
       server. Here, we're just logging the username span of the
       greeting <div> */

        greeting.style.display = "block";
        greetingname.innerHTML = username;

        //clear username input field
        usernameInput.value = "";

        // Optionally, you can submit the form programmatically
        // form.submit();
});
```

Input event

The input event in JavaScript is triggered whenever the value of an input element changes. This event is commonly used with text input fields, textarea elements, and other form elements.

Example 40) In this example:

- We have a form with a password input field and a element to display error messages.
- We use JavaScript to add an event listener for the input event on the password input field.
- When the window browser complete loading, the login button is set to *disabled.*
- Inside the event listener function, we check the length of the password as the user types.
- If the length of the password is less than 8 characters, we display an error message indicating that the password must be at least 8 characters long.
- If the length of the password is 8 characters or more, we clear the error message.

This allows us to provide real-time feedback to the user as they type into the password field, helping them understand the requirements for a valid password.

HTML code

```
<form class="formPassword">
    <label for="password">Password:</label>
    <input type="password" id="password" name="password">
    <p class="passwordmsg">Must be at least 8 characters</p>
    <div id="passwordError"></div>
    <button type="submit" class="loginbtn">Login</button>
 </form>
```

CSS code

```
.formPassword {
      font-family: Arial, Helvetica, sans-serif;
      background-color: azure;
      width: 30%;
      margin: 1em auto;
      padding: 3em 5em;
}
.passwordmsg {
      color: gray;
      font-size: 0.8em;
}
#passwordError {color: red; }
.loginbtn{
      background-color: crimson;
      padding: 0.5em 2em;
      border-radius: 5px;
      border: none;
      color: white;
      cursor: pointer;
 }

.loginbtn:hover { background-color: #333; }

 .loginbtn:disabled{
      background-color: rgba(220, 20, 60, 0.4);
      cursor:not-allowed;
 }
```

JavaScript code

```
// Get reference to the password input element and error message div
const passwordInput = document.querySelector('#password');
const passwordError = document.querySelector('#passwordError');
const loginbtn = document.querySelector('.loginbtn')
```

```
//disabled login button
window.addEventListener("load", function(){
    loginbtn.disabled = true;
})

// Add event listener for the input event on the password input
passwordInput.addEventListener('input', function () {
    // Check the validity of the password as the user types
    const password = passwordInput.value;
       if (password.length < 8) {
           passwordError.textContent = 'Password must be at least 8
           characters long.';
       } else {
           // Clear the error message if the password is valid
            passwordError.textContent = '';
            //enabled login button
            loginbtn.disabled = false;
       }
}
```

Bibliography

Introduction to DOM. (2021). Retrieved from Developer Mozilla.

© 2026. This OER resource is licensed under a Creative Commons license (BY-NC-SA). It may be freely shared and redistributed for non-commercial purposes, provided proper attribution is given and any derivatives are shared under the same license.

Unauthorized copying, reproduction, or distribution of this material, in whole or in part, without permission is not allowed.